

Partie 08 — Corrigé

Mesurer et améliorer les performances

Oskar Laverny

Table des matières

Exercice 1	1
Exercice 2	1
Exercice 3	1
Exercice 4	2
Exercice 5	2
Exercice 6	3
Exercice 7	3
Exercice 8	3
Exercice 9	4

Exercice 1

```
x <- runif(1e7)
system.time(sqrt(x))
```

Les temps fluctuent légèrement selon la charge de la machine, le système et d'autres facteurs. Un benchmark doit donc être répété.

Exercice 2

```
library(bench)
x <- runif(1000)

bench::mark(
  sqrt(x),
  x^0.5
)
```

Il faut comparer les unités, le temps médian, les itérations par seconde et la mémoire allouée. Ne recopiez pas un temps « attendu » : il dépend de la machine et de la version de R.

Exercice 3

```
n <- 5000

bench::mark(
```

```

croissance = {
  res <- numeric(0)
  for (i in 1:n) res <- c(res, i^2)
  res
},
preallocation = {
  res <- numeric(n)
  for (i in seq_len(n)) res[i] <- i^2
  res
},
vectorise = (1:n)^2
)

```

La version qui agrandit continuellement `res` est généralement pénalisée par les allocations répétées. La version vectorisée est très concise pour ce problème.

Exercice 4

```

m <- matrix(runif(1000 * 100), nrow = 1000)

bench::mark(
  apply = apply(m, 2, mean),
  colMeans = colMeans(m)
)

```

`colMeans()` exprime directement l'opération recherchée et utilise une implémentation spécialisée.

Exercice 5

Un exemple possible :

```

lent <- function(x) {
  res <- numeric(length(x))
  for (i in seq_along(x)) {
    m <- mean(x)
    res[i] <- x[i] - m
  }
  res
}

mieux <- function(x) {
  res <- numeric(length(x))
  m <- mean(x)
  for (i in seq_along(x)) {
    res[i] <- x[i] - m
  }
}

```

```
res
}
```

Profilage :

```
x <- runif(1000)
profvis::profvis(lent(x))
```

Benchmark :

```
bench::mark(lent(x), mieux(x))
```

Exercice 6

100×2 microsecondes = 200 microsecondes, soit 0,0002 seconde. Même une amélioration spectaculaire n'aurait quasiment aucun effet sur une analyse de 20 secondes. Il faut chercher un goulot plus important.

Exercice 7

```
x <- c(1, 2, NA, 4)
a <- mean(x, na.rm = TRUE)
b <- sum(x, na.rm = TRUE) / sum(!is.na(x))
all.equal(a, b)
bench::mark(
  moyenne = mean(x, na.rm = TRUE),
  quotient = sum(x, na.rm = TRUE) / sum(!is.na(x))
)
```

Il reste trois observations, pas quatre. Diviser par `length(x)` donne un autre résultat ; un calcul erroné plus rapide n'est pas une optimisation.

Exercice 8

```
croissance <- function(n) {
  res <- numeric(0)
  for (i in seq_len(n)) res <- c(res, i^2)
  res
}
preallocation <- function(n) {
  res <- numeric(n)
  for (i in seq_len(n)) res[i] <- i^2
  res
}
for (n in c(100, 1000, 5000)) {
  print(n)
  print(bench::mark(
```

```

    croissance = croissance(n),
    preallocation = preallocation(n),
    vectorise = seq_len(n)^2
  ))
}

```

Les rapports dépendent de la taille et des coûts fixes. Les copies répétées de la version par croissance deviennent plus coûteuses quand n augmente.

Exercice 9

```

lent <- function(x) {
  res <- numeric(length(x))
  for (i in seq_along(x)) res[i] <- x[i] - mean(x)
  res
}
mieux <- function(x) {
  res <- numeric(length(x))
  m <- mean(x)
  for (i in seq_along(x)) res[i] <- x[i] - m
  res
}
x <- runif(1000)
stopifnot(isTRUE(all.equal(lent(x), mieux(x))))
stopifnot(isTRUE(all.equal(mieux(x), x - mean(x))))
bench::mark(lent(x), mieux(x), x - mean(x))

```

On supprime le recalcul de la même moyenne : la première méthode effectue un travail qui croît quadratiquement avec la longueur du vecteur, alors que les deux autres effectuent un travail linéaire.